

# Heavy-Light Decomposition

# Heavy-light decomposition

## Fontes de consulta

- <https://cp-algorithms.com/graph/hld.html> (algumas figuras foram extraídas daqui).
- <https://codeforces.com/blog/entry/53170>
- <https://usaco.guide/plat/hld?lang=cpp>

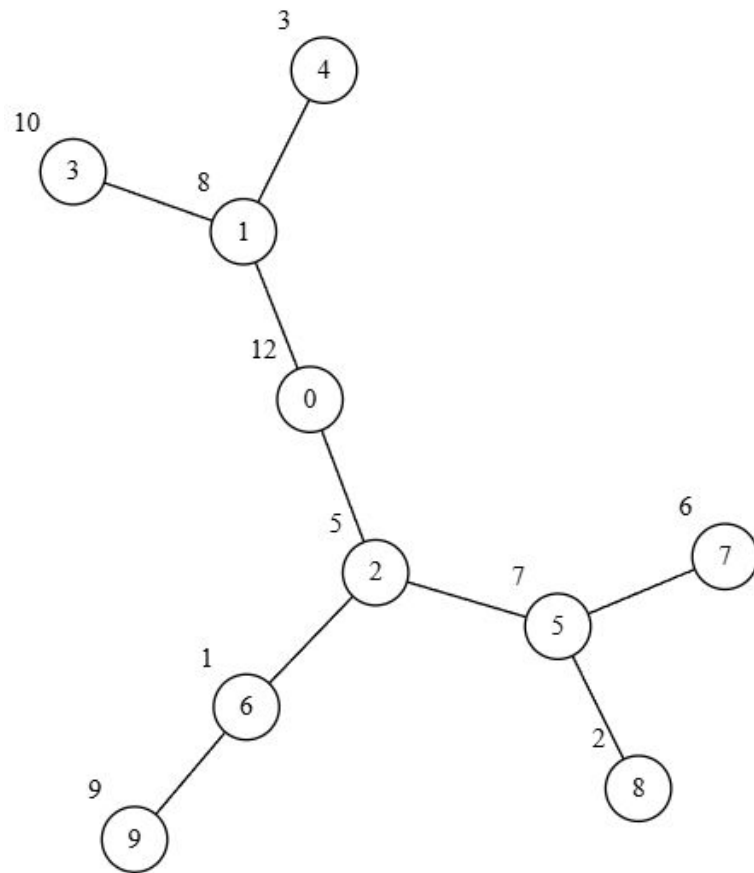
Aluno do mestrado ajudando na disciplina:

- Samuel Raimundo, email: [samuel.r.pinto@ufv.br](mailto:samuel.r.pinto@ufv.br)

# Heavy-light decomposition

## Motivação:

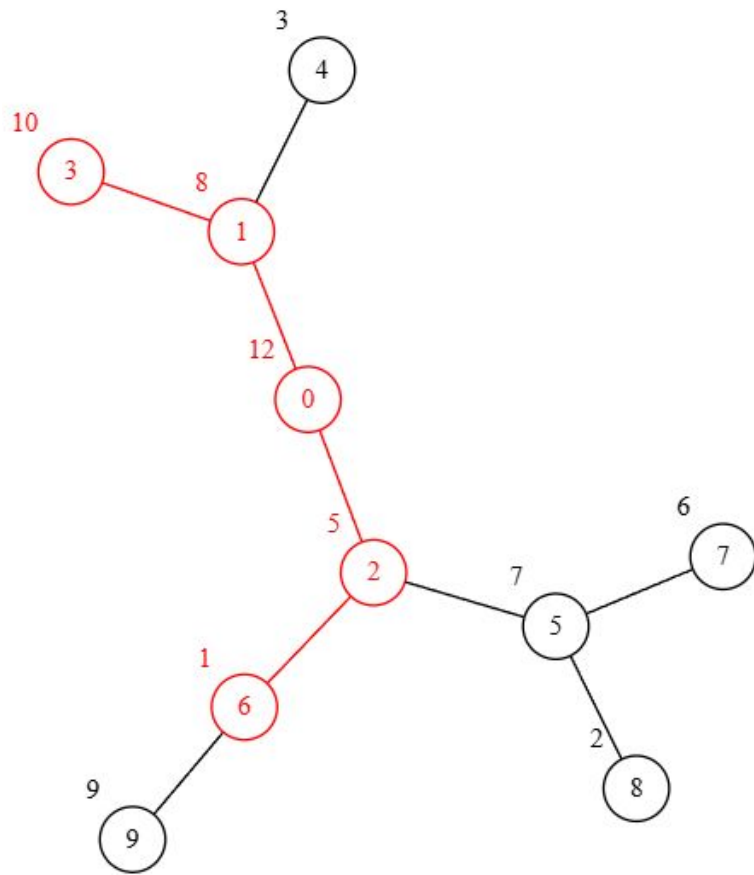
- Uma seg-tree permite fazer alterações e consultas eficientes em intervalos.
- Como fazer o mesmo em uma árvore?
- Exemplo:
  - Vértices (ou arestas) possuem valores
  - Menor valor no caminho 3-6?
  - Permitir consultas (menor, maior, soma, etc)
  - Permitir alterações (soma, troca, etc) em vértices e/ou em caminhos inteiros.
  - Tempo desejado: sublinear



# Heavy-light decomposition

## Motivação:

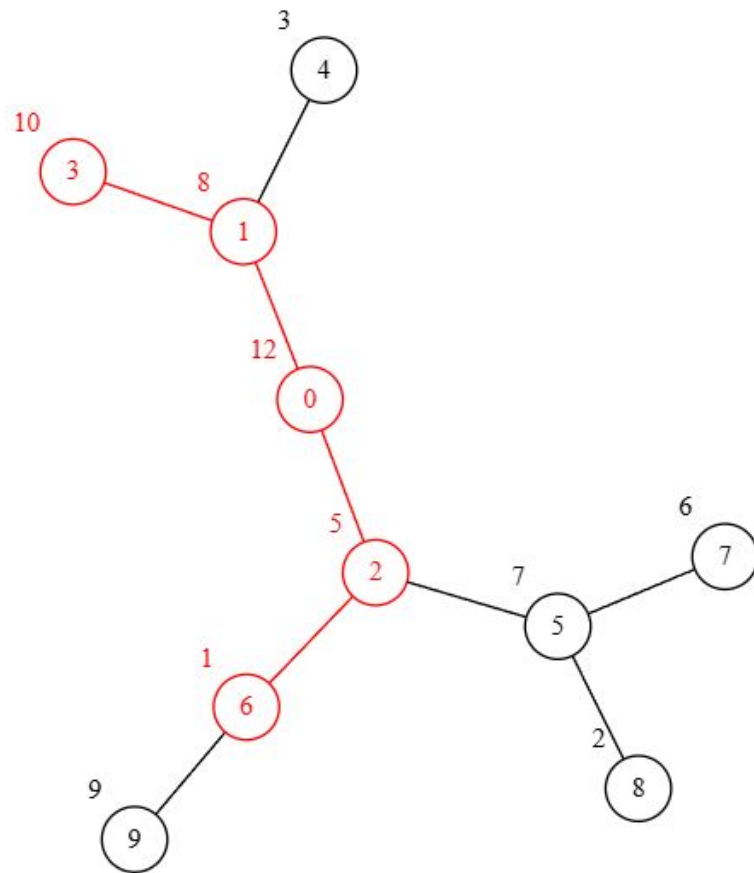
- Uma seg-tree permite fazer alterações e consultas eficientes em intervalos.
- Como fazer o mesmo em uma árvore?
- Exemplo:
  - Vértices (ou arestas) possuem valores
  - Menor valor no caminho 3-6?
  - Permitir consultas (menor, maior, soma, etc)
  - Permitir alterações (soma, troca, etc) em vértices e/ou em caminhos inteiros.
  - Tempo desejado: sublinear
  - $\text{Min}(3,6) =$



# Heavy-light decomposition

## Motivação:

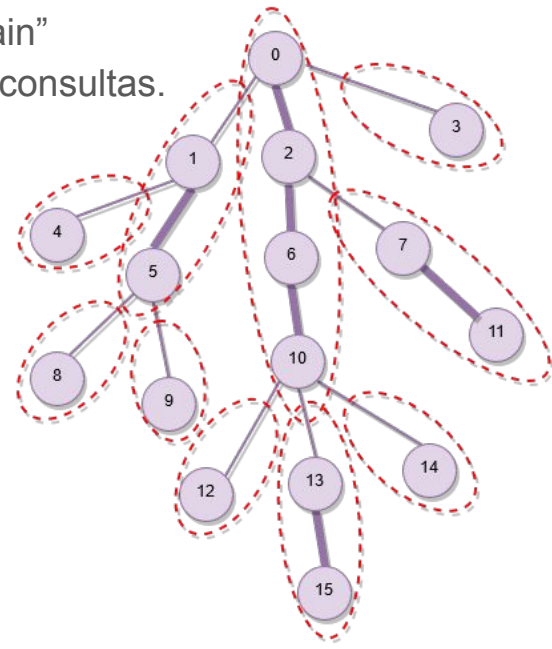
- Uma seg-tree permite fazer alterações e consultas eficientes em intervalos.
- Como fazer o mesmo em uma árvore?
- Exemplo:
  - Vértices (ou arestas) possuem valores
  - Menor valor no caminho 3-6?
  - Permitir consultas (menor, maior, soma, etc)
  - Permitir alterações (soma, troca, etc) em vértices e/ou em caminhos inteiros.
  - Tempo desejado: sublinear
  - $\text{Min}(3,6) = 1$
  - Se valores são distância a uma raiz → encontra LCA



# Heavy-light decomposition

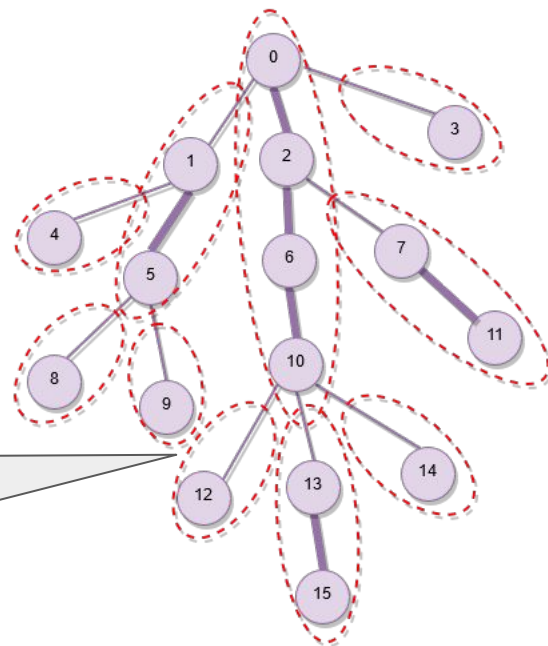
- Ideia principal: dividir árvore em vários caminhos/chains de modo que:
  - Seja possível atingir a raiz passando por no máximo  $O(\log n)$  chains.
    - Para trocar de chain precisamos passar por uma aresta leve. A subárvore após cada aresta leve tem menos da metade do tamanho da árvore original.
  - Nenhuma chain deve interceptar outra.
  - Permite responder a consultas do tipo “calcular algo no caminho de a até b” fazendo várias consultas do tipo “calcule algo no segmento  $[l,r]$  da k-ésima chain”
  - Em cada uma das chains, teremos uma seg-tree para fazer as consultas.
  - No exemplo ao lado:
    - $\text{Min}(5,6) = \min( \text{Min}(5,1), \text{Min}(0,6) )$

\* usaremos o termo chain para não confundir com o caminho entre os vértices da consulta.



# Conceitos

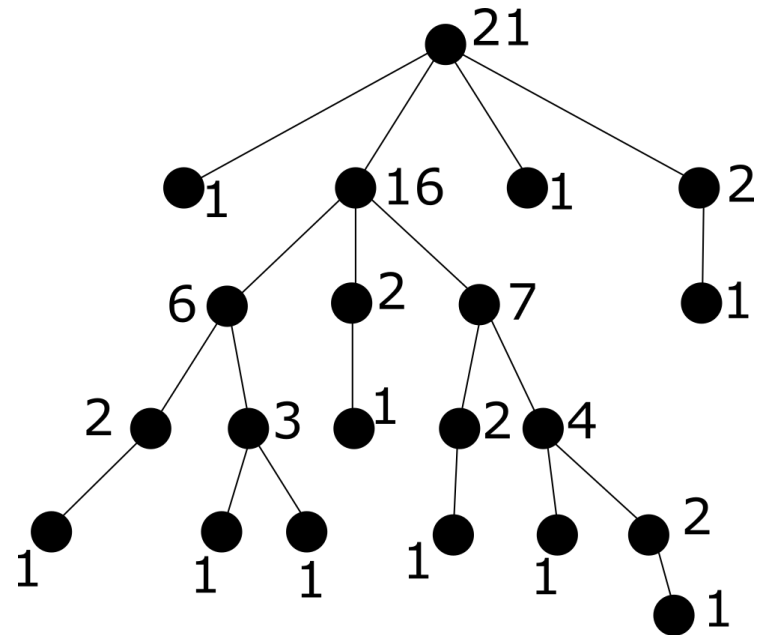
- Escolhe-se um nodo da árvore para virar raiz.
- Filho **pesado** de um nodo: o filho cuja subárvore é maior (exemplo: 2 é o filho pesado de 0). Algumas implementações usam ideias um pouco diferentes.
- Filho **leve**: filho não pesado
- Aresta **pesada**: conecta um nodo ao filho pesado
- Aresta **leve**: conecta um nodo a outro filho
- A árvore é dividida em várias chains formadas pelas arestas pesadas
  - Pode-se provar que o caminho de um vértice até a raiz passa por no máximo  $\log n$  chains distintas



a aresta (10,13) deveria ser pesada, pela definição acima (esta figura foi feita usando uma definição um pouco diferente)

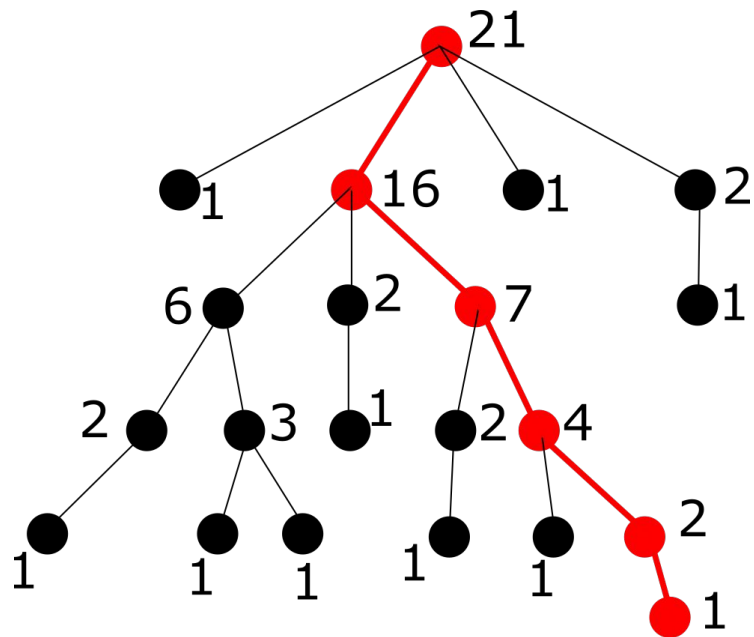
# Decompondo a árvore em chains

- A decomposição: baseada no tamanho da sub-árvore



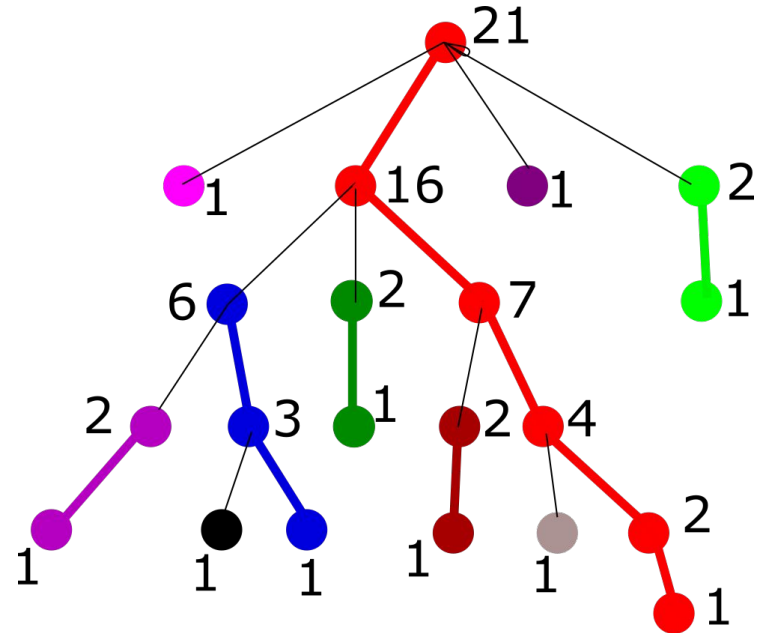
# Decompondo a árvore em chains

- A decomposição: baseada no tamanho da sub-árvore
  - Segue o maior filho



# Decompondo a árvore em chains

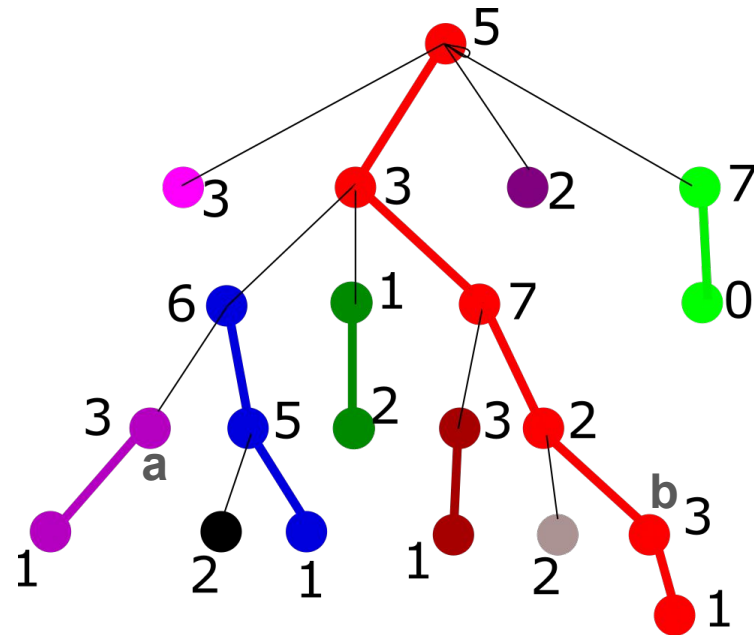
- A decomposição: baseada no tamanho da sub-árvore
  - Segue o maior filho
- No exemplo ao lado: nós pesados estão mais espessos.



# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $\text{Query}(a,b) = \min( \text{Query}(a,\text{lca}) , \text{Query}(b,\text{lca}) )$  , onde  $\text{lca} = \text{lca}(a,b)$

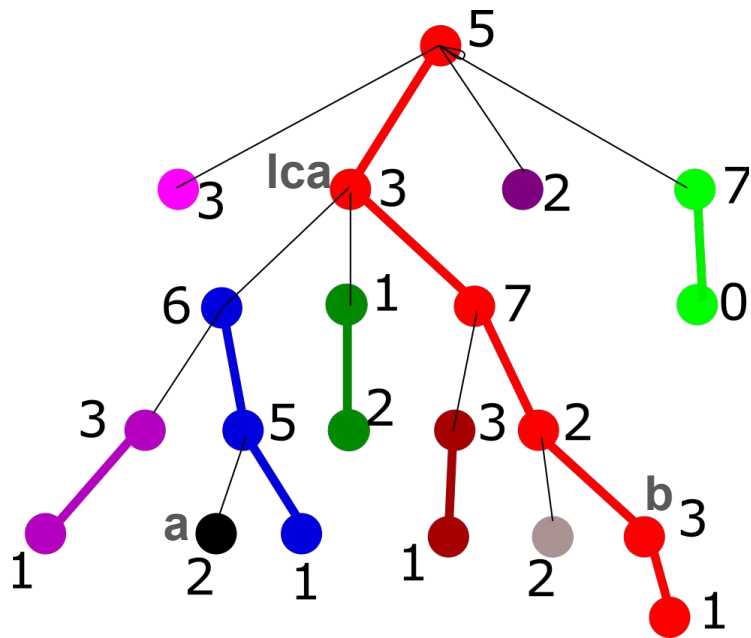
\* Agora, assuma que números são os valores dos vértices



# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $Query(a,b) = \min( Query(a,lca) , Query(b,lca) )$  , onde  $lca = lca(a,b)$
- $Query(v, lca) =$  (vale para lca ou qualquer ancestral de v)
  - $ans = \text{infinito}$
  - $\text{while}( chain(v) \neq chain(lca) )$ 
    - $ans = \min(ans, \dots )$

\* Agora, assuma que números são os valores dos vértices

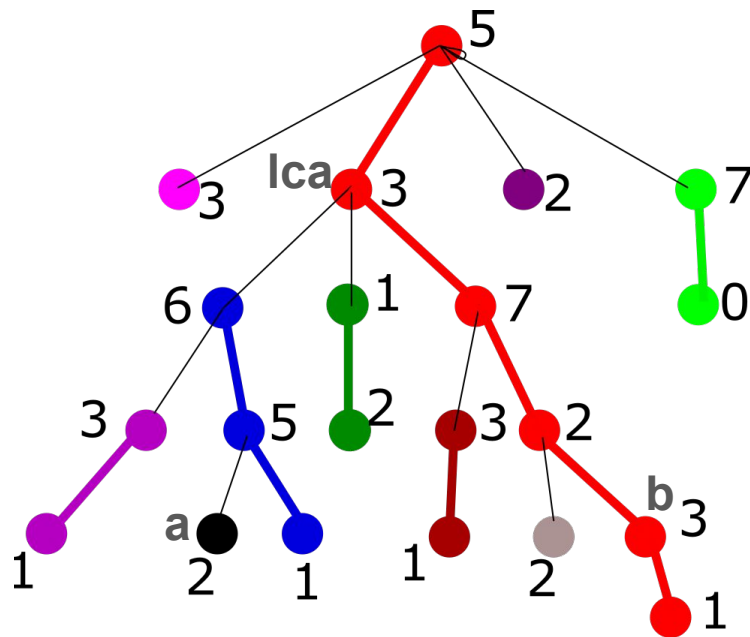


# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $\text{Query}(a,b) = \min( \text{Query}(a,\text{lca}) , \text{Query}(b,\text{lca}) )$  , onde  $\text{lca} = \text{lca}(a,b)$
- $\text{Query}(v, \text{lca}) =$  (vale para lca ou qualquer ancestral de v)
  - $\text{ans} = \text{infinito}$
  - $\text{while}( \text{chain}(v) \neq \text{chain}(\text{lca}) )$ 
    - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{head}(\text{chain}(v)) ) )$
    - $v = \text{head}(\text{chain}(v)) \rightarrow \text{parent}$

seg tree da chain de v

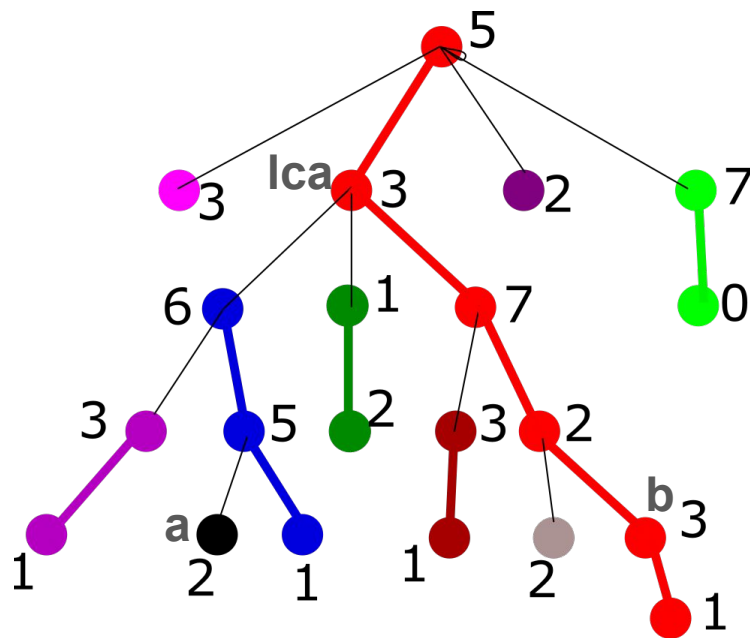
\* Agora, assuma que números são os valores dos vértices



# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $\text{Query}(a,b) = \min( \text{Query}(a,\text{lca}) , \text{Query}(b,\text{lca}) )$  , onde  $\text{lca} = \text{lca}(a,b)$
- $\text{Query}(v, \text{lca}) =$  (vale para lca ou qualquer ancestral de v)
  - $\text{ans} = \text{infinito}$
  - $\text{while}( \text{chain}(v) \neq \text{chain}(\text{lca}) )$ 
    - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{head}(\text{chain}(v)) ) )$
    - $v = \text{head}(\text{chain}(v)) \rightarrow \text{parent}$
  - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{lca}) )$

\* Agora, assuma que números são os valores dos vértices

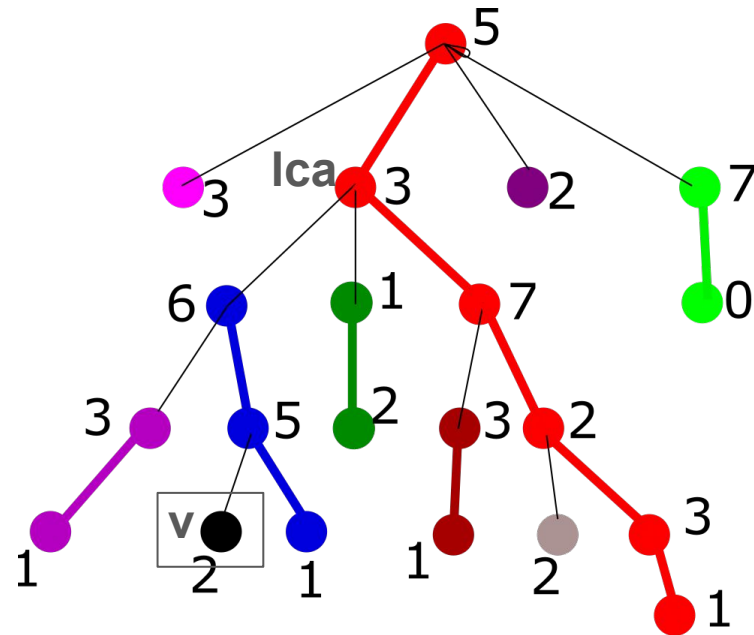


# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $\text{Query}(a,b) = \min( \text{Query}(a,\text{lca}) , \text{Query}(b,\text{lca}) )$  , onde  $\text{lca} = \text{lca}(a,b)$
- $\text{Query}(v, \text{lca}) =$  (vale para lca ou qualquer ancestral de v)
  - **ans = infinito**
  - while( chain(v) != chain(lca) )
    - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{head}(\text{chain}(v)) ) )$
    - $v = \text{head}(\text{chain}(v)) \rightarrow \text{parent}$
  - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{lca}) )$

ans = inf

\* Agora, assuma que números são os valores dos vértices

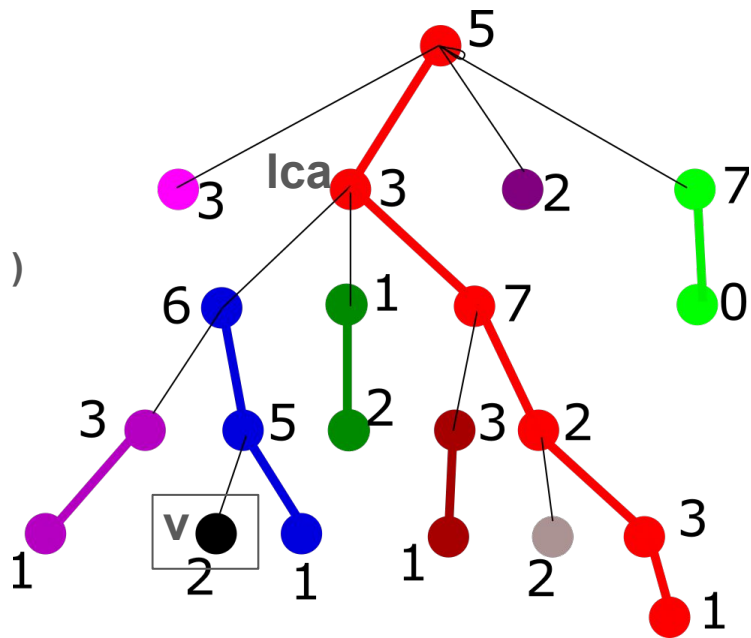


# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $\text{Query}(a,b) = \min( \text{Query}(a,\text{lca}) , \text{Query}(b,\text{lca}) )$  , onde  $\text{lca} = \text{lca}(a,b)$
- $\text{Query}(v, \text{lca}) =$  (vale para lca ou qualquer ancestral de v)
  - $\text{ans} = \text{infinito}$
  - $\text{while}( \text{chain}(v) \neq \text{chain}(\text{lca}) )$ 
    - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{head}(\text{chain}(v)) ) )$
    - $v = \text{head}(\text{chain}(v)) \rightarrow \text{parent}$
  - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{lca}) )$

$\text{ans} = 2$

\* Agora, assuma que números são os valores dos vértices

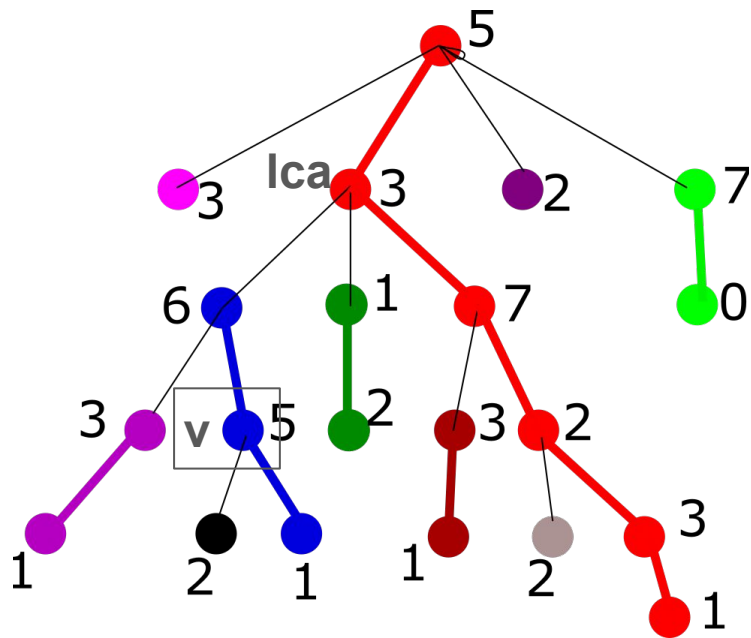


# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $\text{Query}(a,b) = \min( \text{Query}(a,\text{lca}) , \text{Query}(b,\text{lca}) )$  , onde  $\text{lca} = \text{lca}(a,b)$
- $\text{Query}(v, \text{lca}) =$  (vale para lca ou qualquer ancestral de v)
  - $\text{ans} = \text{infinito}$
  - $\text{while}( \text{chain}(v) \neq \text{chain}(\text{lca}) )$ 
    - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{head}(\text{chain}(v)) ) )$
    - **$v = \text{head}(\text{chain}(v)) \rightarrow \text{parent}$**
  - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{lca}) )$

$\text{ans} = 2$

\* Agora, assuma que números são os valores dos vértices

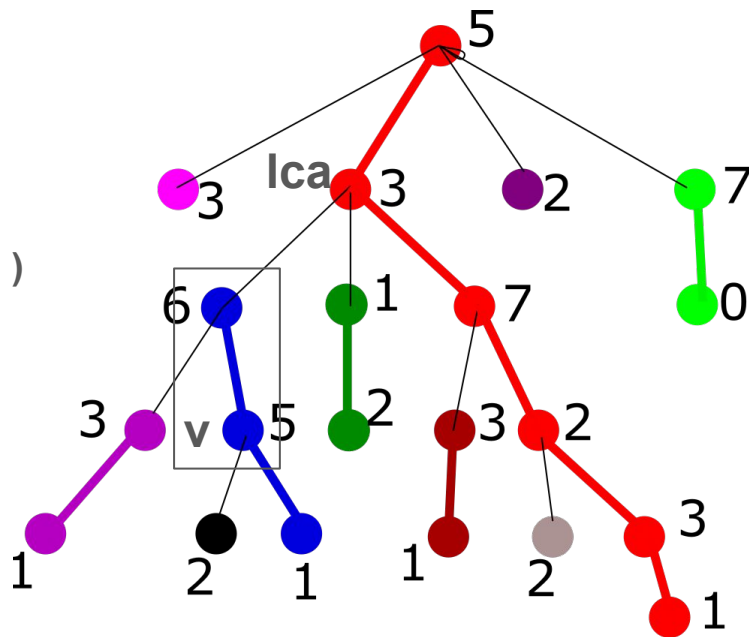


# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $\text{Query}(a,b) = \min( \text{Query}(a,\text{lca}) , \text{Query}(b,\text{lca}) )$  , onde  $\text{lca} = \text{lca}(a,b)$
- $\text{Query}(v, \text{lca}) =$  (vale para lca ou qualquer ancestral de v)
  - $\text{ans} = \text{infinito}$
  - $\text{while}( \text{chain}(v) \neq \text{chain}(\text{lca}) )$ 
    - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{head}(\text{chain}(v)) ) )$
    - $v = \text{head}(\text{chain}(v)) \rightarrow \text{parent}$
  - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{lca}) )$

$$\text{ans} = \min(2, 5) = 2$$

\* Agora, assuma que números são os valores dos vértices

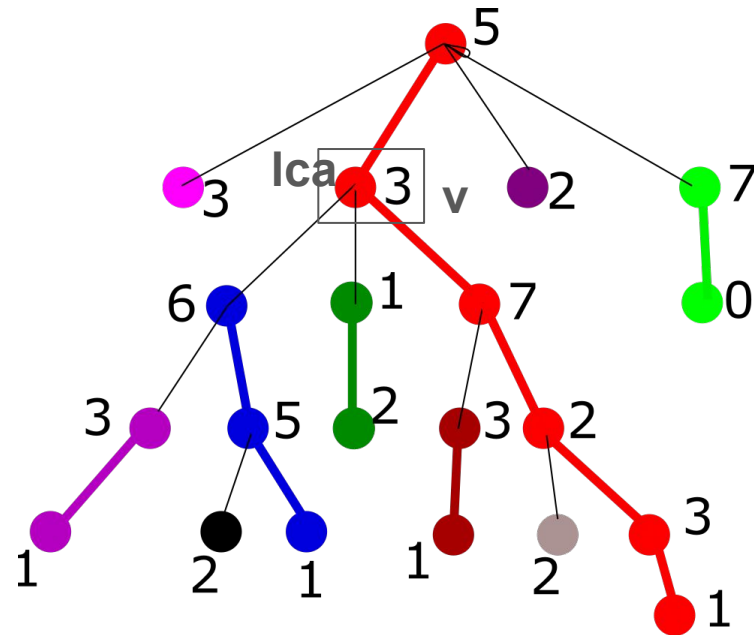


# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $\text{Query}(a,b) = \min( \text{Query}(a,\text{lca}) , \text{Query}(b,\text{lca}) )$  , onde  $\text{lca} = \text{lca}(a,b)$
- $\text{Query}(v, \text{lca}) =$  (vale para lca ou qualquer ancestral de v)
  - $\text{ans} = \text{infinito}$
  - $\text{while}( \text{chain}(v) \neq \text{chain}(\text{lca}) )$ 
    - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{head}(\text{chain}(v)) ) )$
    - **$v = \text{head}(\text{chain}(v)) \rightarrow \text{parent}$**
  - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{lca}) )$

$\text{ans} = 2$

\* Agora, assuma que números são os valores dos vértices

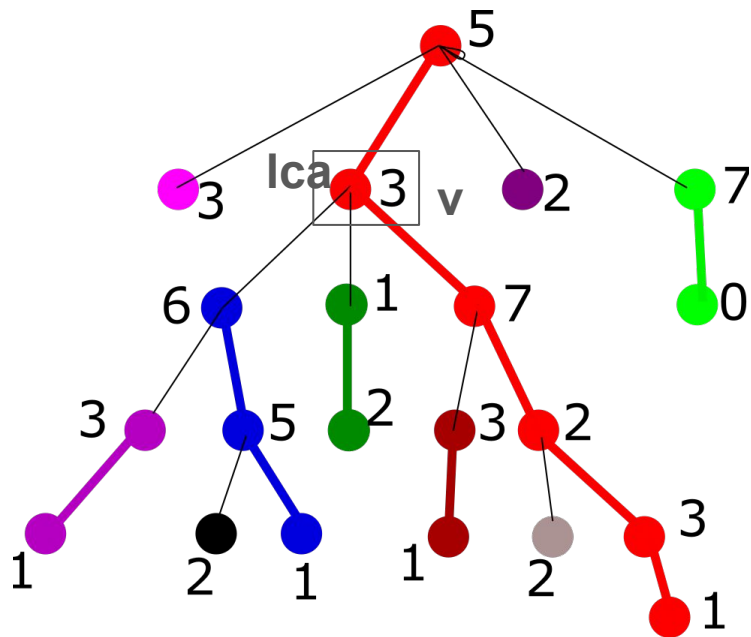


# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $\text{Query}(a,b) = \min( \text{Query}(a,\text{lca}) , \text{Query}(b,\text{lca}) )$  , onde  $\text{lca} = \text{lca}(a,b)$
- $\text{Query}(v, \text{lca}) =$  (vale para lca ou qualquer ancestral de v)
  - $\text{ans} = \text{infinito}$
  - $\text{while}( \text{chain}(v) \neq \text{chain}(\text{lca}) )$ 
    - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{head}(\text{chain}(v)) ) )$
    - $v = \text{head}(\text{chain}(v)) \rightarrow \text{parent}$
  - **$\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{lca}) )$**

$$\text{ans} = \min(2,3) = 2$$

\* Agora, assuma que números são os valores dos vértices



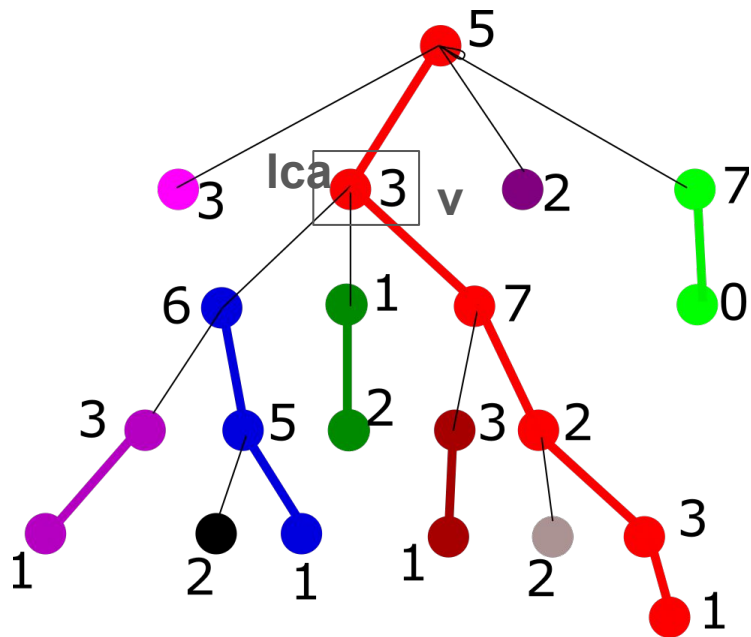
# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain
- $\text{Query}(a,b) = \min( \text{Query}(a,\text{lca}) , \text{Query}(b,\text{lca}) )$  , onde  $\text{lca} = \text{lca}(a,b)$
- $\text{Query}(v, \text{lca}) =$  (vale para lca ou qualquer ancestral de v)
  - $\text{ans} = \text{infinito}$
  - $\text{while}( \text{chain}(v) \neq \text{chain}(\text{lca}) )$ 
    - $\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{head}(\text{chain}(v)) ) )$
    - $v = \text{head}(\text{chain}(v)) \rightarrow \text{parent}$
  - **$\text{ans} = \min(\text{ans}, \text{seg.query}(v, \text{lca}) )$**

$$\text{ans} = \min(2,3) = 2$$

Poderia ser  
 $\text{query}(\text{lca}, \text{lca})$  ?

\* Agora, assuma que números são os valores dos vértices

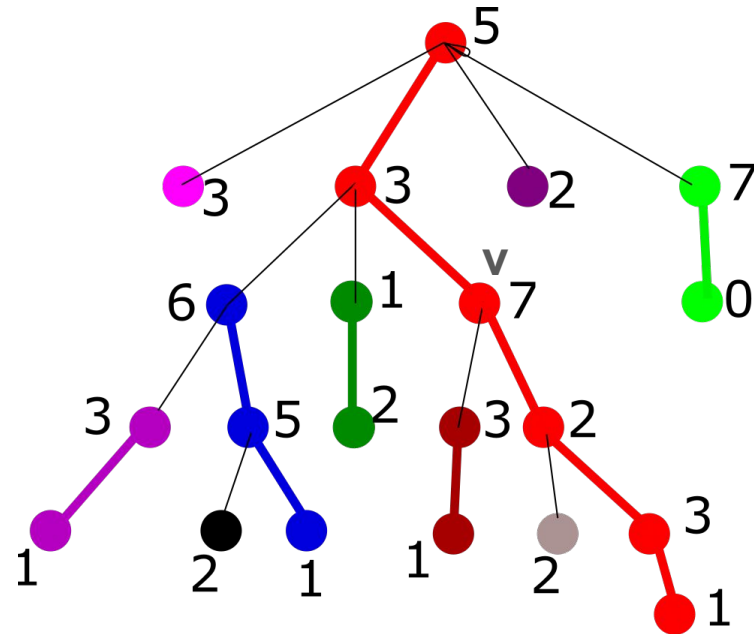


# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain

- E se quisermos trocar o valor de  $v$  de 7 para 1 ?
- $\text{Update}(v, \text{valor}) =$

\* Agora, assuma que números são os valores dos vértices



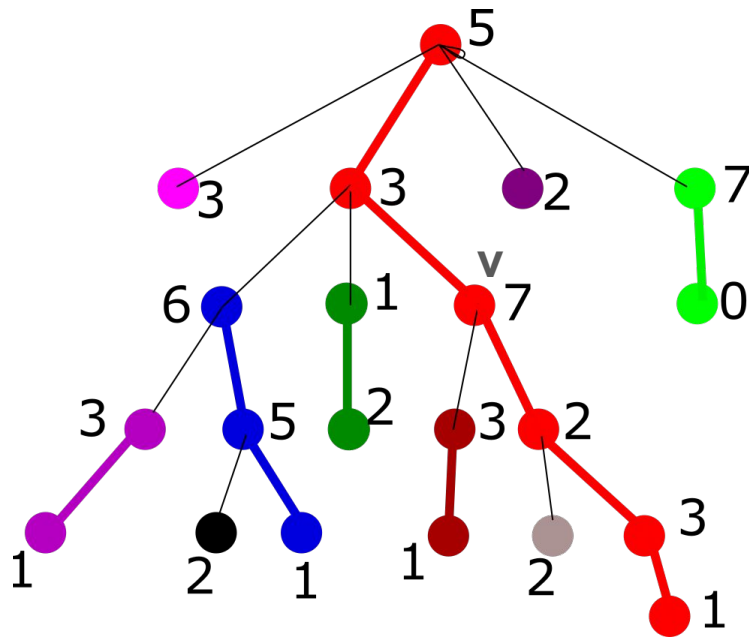
# Algoritmo

- Criar árvore e escolher raiz
- Decompor em chains, colocando uma seg-tree em cada chain

- E se quisermos trocar o valor de v de 7 para 1 ?
- `Update(v, valor) = seg.update(v, valor)`

seg tree da chain  
contendo v

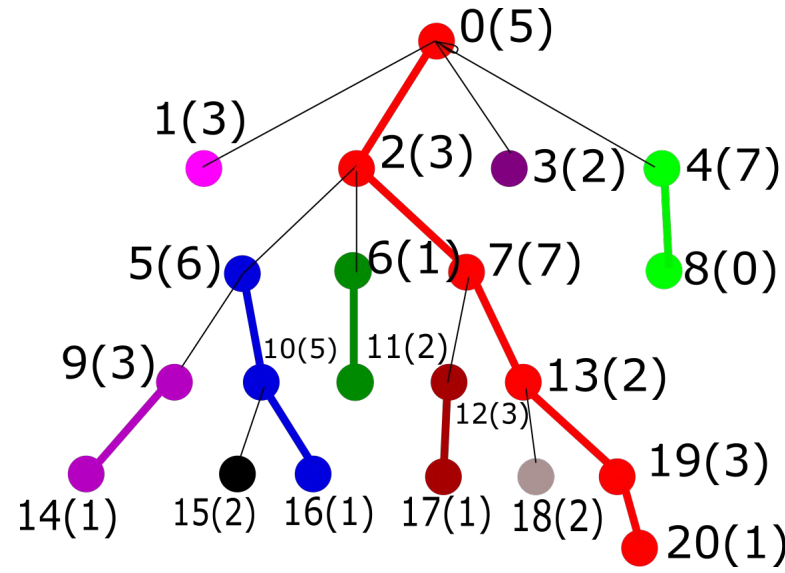
\* Agora, assuma que números são os valores dos vértices



# Implementação

- Em geral, a implementação é simplificada utilizando algumas estratégias.
- Primeiro: filho pesado é colocado na frente da lista de adjacência de cada vértice.
- DFS 1: calcula tamanho de subárvores, reorganiza filhos.

```
//prev = nodo anterior na DFS (pai)
void dfs(vector<vector<int> > &adj,int root, int prev) {
    sz[root] = 1;
    //w TEM que ser por referencia (por causa do swap!!!)
    for(int &w:adj[root]) if(w!=prev) {
        depth[w] = depth[root] + 1; //opcional
        dfs(adj,w,root);
        sz[root] += sz[w];
        //o primeiro filho de cada vértice será sempre o maior
        //(ou seja, terá um heavy edge entre eles)
        if(sz[w] > sz[ adj[root][0] ] || adj[root][0]==prev)
            swap(adj[root][0],w);
    }
}
```

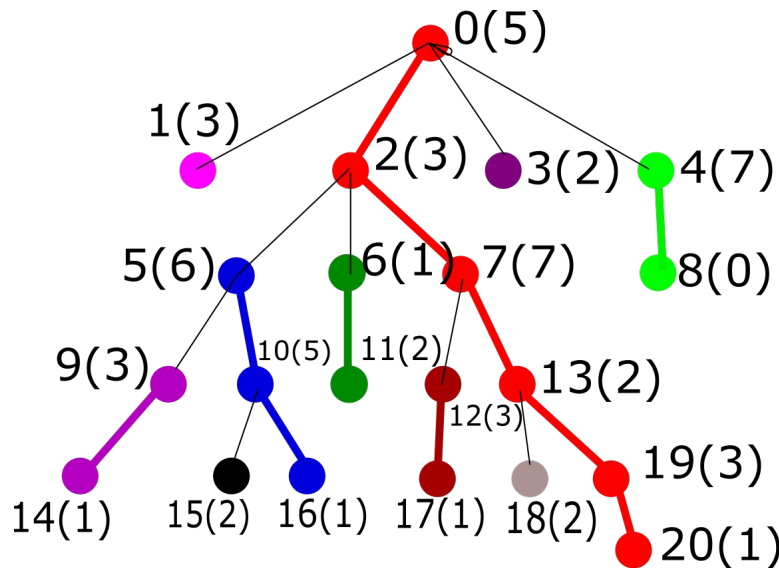


# Implementação

- Ordem da DFS:

| ord       | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | ... |
|-----------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|-----|
| vert      | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | ... |
| v (valor) | 5 | 3 | 7 | 2  | 3  | 1  | 2  | 3  | 1  | 1 | 2  | 6  | 5  | 1  | 2  | 3  | 1  | 2  | ... |

- Qual a consequência disso?
  - Podemos guardar tudo em uma única seg-tree
  - Exemplo:
    - Query(15,0) = min:
      - seg.query(15,15)
      - seg.query(10,5)
      - seg.query(2,0)

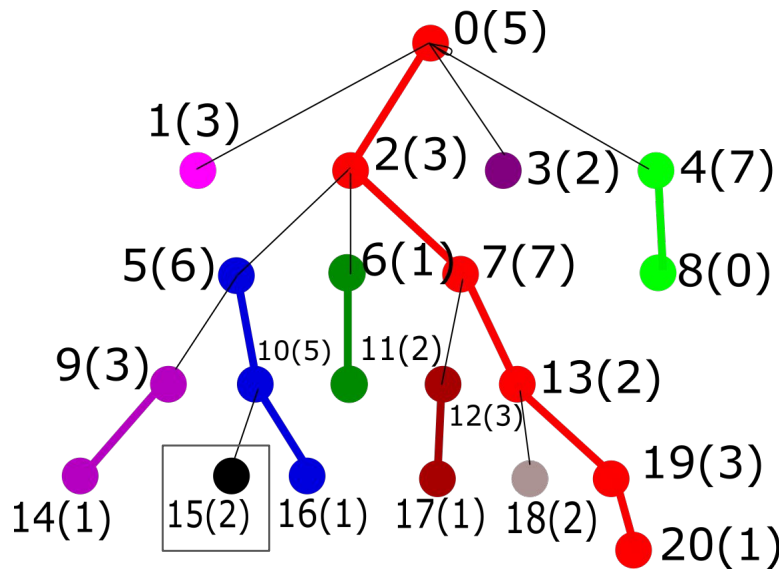


# Implementação

- Ordem da DFS:

| ord       | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | ... |
|-----------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|-----|
| vert      | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | ... |
| v (valor) | 5 | 3 | 7 | 2  | 3  | 1  | 2  | 3  | 1  | 1 | 2  | 6  | 5  | 1  | 2  | 3  | 1  | 2  | ... |

- Qual a consequência disso?
  - Podemos guardar tudo em uma única seg-tree
  - Exemplo:
    - Query(15,0) = min:
      - **seg.query(15,15)**
      - seg.query(10,5)
      - seg.query(2,0)



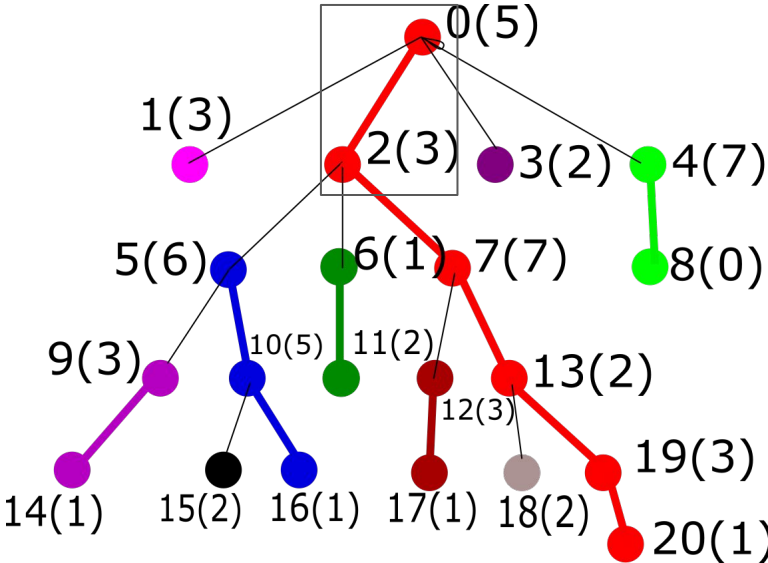


# Implementação

- Ordem da DFS:

|           |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |     |
|-----------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|-----|
| ord       | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | ... |
| vert      | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | ... |
| v (valor) | 5 | 3 | 7 | 2  | 3  | 1  | 2  | 3  | 1  | 1 | 2  | 6  | 5  | 1  | 2  | 3  | 1  | 2  | ... |

- Qual a consequência disso?
  - Podemos guardar tudo em uma única seg-tree
  - Exemplo:
    - Query(15,0) = min:
      - seg.query(15,15)
      - seg.query(10,5)
      - **seg.query(2,0)**



# Implementação

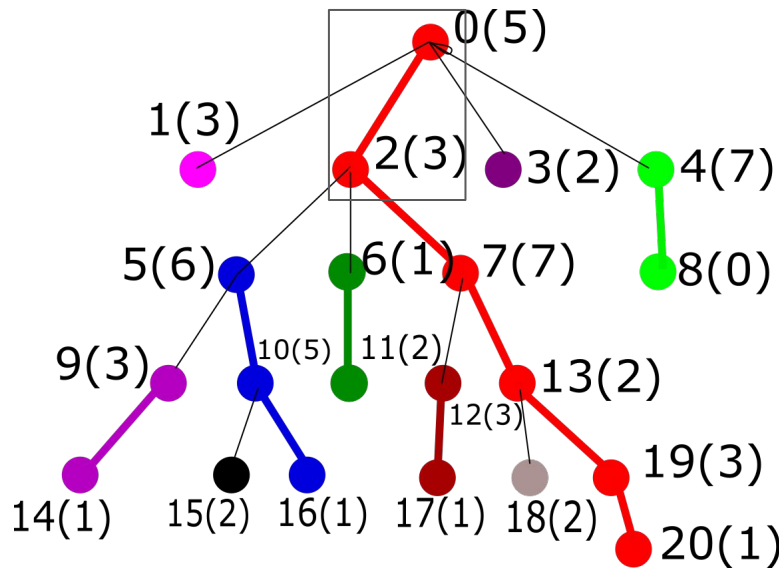
- Ordem da DFS:

|           |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |     |
|-----------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|-----|
| ord       | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | ... |
| vert      | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | ... |
| v (valor) | 5 | 3 | 7 | 2  | 3  | 1  | 2  | 3  | 1  | 1 | 2  | 6  | 5  | 1  | 2  | 3  | 1  | 2  | ... |

- Qual a consequência disso?
  - Podemos guardar tudo em uma única seg-tree
  - Exemplo:

- Query(15,0) = min:
  - `seg.query(15,15) → seg.query(14,14)`
  - `seg.query(10,5) → seg.query(11,12)`
  - **`seg.query(2,0) → seg.query(0,1)`**

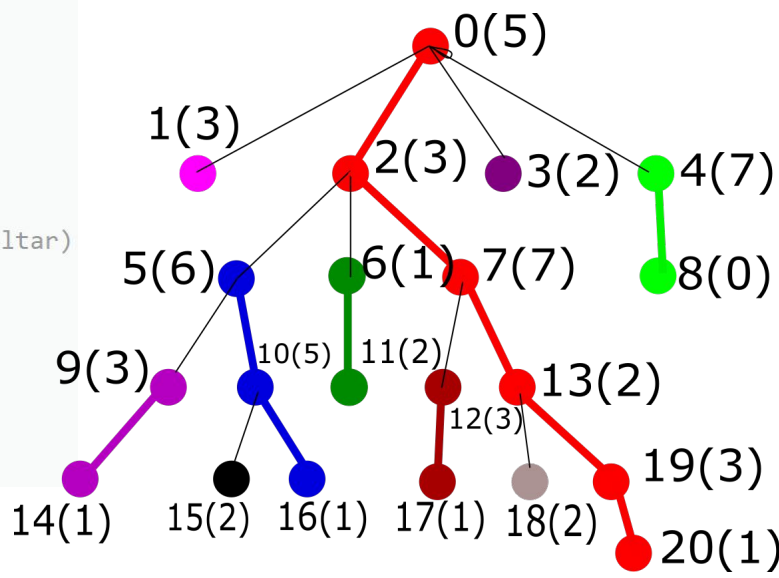
precisamos calcular onde  
(nos vetores acima) cada  
vértice está



# Implementação

- Em geral, a implementação é simplificada utilizando algumas estratégias.
- Primeiro: filho pesado é colocado na frente da lista de adjacência de cada vértice.
- DFS 1: calcula tamanho de subárvores, reorganiza filhos.
- DFS 2: calcula os vetores dos slides anteriores

```
//ct = ordem de visitacao dos vertices (comeca de 0)
//value = peso das arestas, que ficara na seg tree
void build(vector<vector<int> > &adj, vector<T> &v,
          int &ct, int root, int prev) {
    pos[root] = ct; //onde cada vértice está na ordem da DFS?
    v[ct] = value[root]; //valor de cada vértice (na ordem da dfs)
    ct++;
    for(int w:adj[root]) if(w!=prev) { //vizinhos de root (cuidado para nao voltar)
        parent[w] = root; //pai de cada vértice (para subir)
        //cada vertice é cabeça da sua chain.
        //Depois arrumamos isso para os vertices que estiverem no heavy path
        head[w] = (w==adj[root][0]?head[root]:w);
        build(v,ct,w,root);
    }
}
```



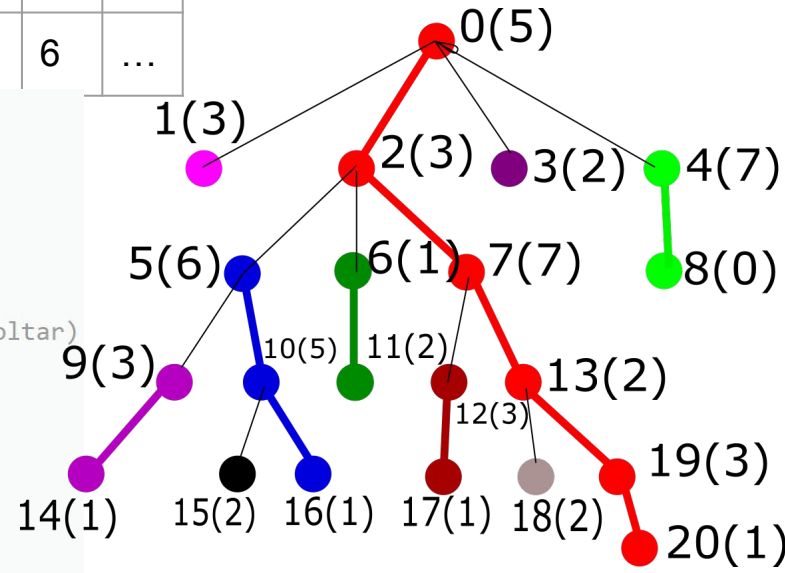
|      |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |     |
|------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|-----|
| ord  | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | ... |
| vert | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | ... |
| v    | 5 | 3 | 7 | 2  | 3  | 1  | 2  | 3  | 1  | 1 | 2  | 6  | 5  | 1  | 2  | 3  | 1  | 2  | ... |

|        |    |   |   |    |   |    |   |   |   |    |    |    |     |
|--------|----|---|---|----|---|----|---|---|---|----|----|----|-----|
|        | 0  | 1 | 2 | 3  | 4 | 5  | 6 | 7 | 8 | 9  | 10 | 11 | ... |
| pos    | 0  | ? | 1 | 17 | ? | 11 | 9 | 2 | ? | 15 | 12 | 10 | ... |
| parent | -1 | 0 | 0 | 0  | 0 | 2  | 2 | 2 | 4 | 5  | 5  | 6  | ... |
| head   | 0  | 1 | 0 | 3  | 4 | 5  | 6 | 0 | 4 | 9  | 5  | 6  | ... |

```

//ct = ordem de visitacao dos vertices (comeca de 0)
//value = peso das arestas, que ficara na seg tree
void build(vector<vector<int> > &adj, vector<T> &v,
           int &ct, int root, int prev) {
    pos[root] = ct; //onde cada vértice está na ordem da DFS?
    v[ct] = value[root]; //valor de cada vértice (na ordem da dfs)
    ct++;
    for(int w:adj[root]) if(w!=prev) { //vizinhos de root (cuidado para nao voltar)
        parent[w] = root; //pai de cada vértice (para subir)
        //cada vertice é cabeca da sua chain.
        //Depois arrumamos isso para os vertices que estiverem no heavy path
        head[w] = (w==adj[root][0]?head[root]:w);
        build(v, ct, w, root);
    }
}

```



# Implementação

- Em geral, a implementação é simplificada utilizando algumas estratégias.
- Primeiro: filho pesado é colocado na frente da lista de adjacência de cada vértice.
- DFS 1: calcula tamanho de subárvores, reorganiza filhos.
- DFS 2: calcula os vetores dos slides anteriores
- Construtor:

```
Hld(vector<vector<int> > &adj, vector<T> &value_, int root=0): st(adj.size()) {  
  
    ct = 0; //marca os vertices com codigo 0,1,2 ... com base na posicao em que vao ficar na seg tree  
    int n = adj.size();  
    pos = sz = parent = head = depth = vector<int>(n);  
    value = value_;  
  
    vector<int> v(n); //vetor com pesos dos vertices na seg tree..  
    head[root] = root;  
    depth[root] = 0; //vamos considerar que a raiz esta na profundidade 0 (opcional)  
    dfs(adj, root, -1);  
    build(adj, v, ct, root, -1); //segunda DFS, após colocar as arestas pesadas como (u, adj[u][0])  
    st.build(v); //cria a Seg Tree  
}
```

|      |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |    |    |    |
|------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|----|----|----|
| ord  | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| vert | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | 4  | 8  | 1  |

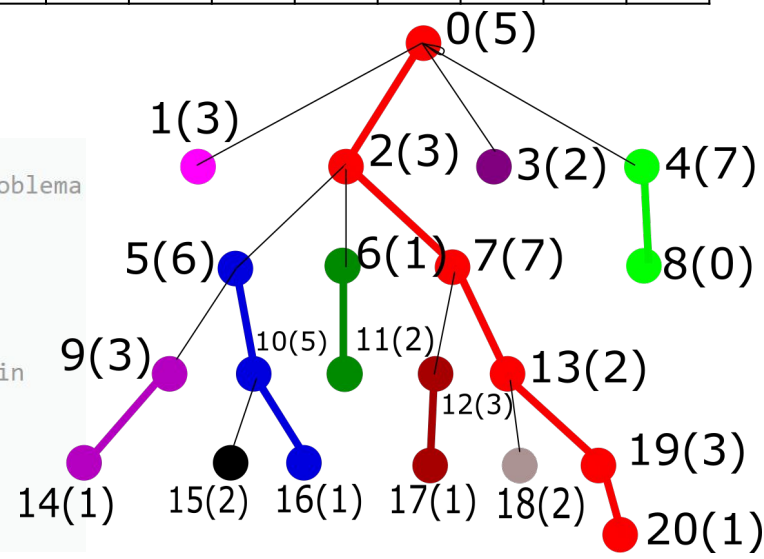
|     |    |    |   |    |    |    |   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----|----|----|---|----|----|----|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|
| ord | 0  | 1  | 2 | 3  | 4  | 5  | 6 | 7 | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| pos | 0  | 20 | 1 | 17 | 18 | 11 | 9 | 2 | 19 | 15 | 12 | 10 | 7  | 3  | 16 | 14 | 13 | 8  | 6  | 4  | 5  |
| par | -1 | 0  | 0 | 0  | 0  | 2  | 2 | 2 | 4  | 5  | 5  | 6  | 7  | 7  | 9  | 10 | 10 | 12 | 13 | 13 | 19 |
| hea | 0  | 1  | 0 | 3  | 4  | 5  | 6 | 0 | 4  | 9  | 5  | 6  | 12 | 0  | 9  | 15 | 5  | 12 | 18 | 0  | 0  |

O primeiro if: u estará sempre após v na dfs

- Se u e v na mesma chain → u estará abaixo
- Se em chains diferentes → a chain de u estará abaixo (exemplo: u=5, v = 19)

```
//as consultas nas chains sao sempre de cima para baixo
//note que se o pai de head de v estiver no meio da chain de u --> nao gera problema
//pois o pos de u será menor que v e trocariamos os dois..
//ou seja, sempre teremos pos[u] >= pos[v]
//assim, a consulta é sempre do pos maior para o menor...
T query(int u, int v) { //consulta nos nodos do caminho entre vertices u e v
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) return st.query(pos[v], pos[u]); //estao na mesma chain

    //ATUALIZAR (exemplo: minimo de caminho, maximo de caminho, etc)
    //implementacao atual: soma
    return st.query(pos[head[u]],pos[u]) + query(parent[head[u]], v) ;
}
```



|      |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |    |    |    |
|------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|----|----|----|
| ord  | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| vert | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | 4  | 8  | 1  |

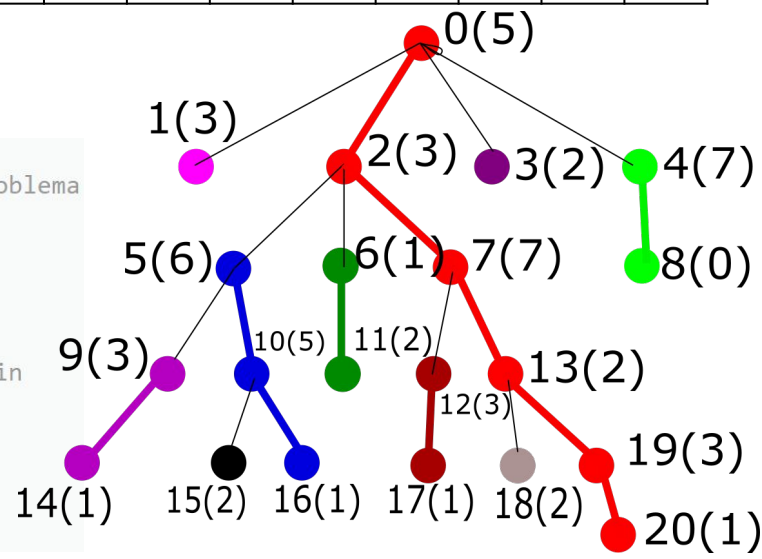
  

|     |    |    |   |    |    |    |   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----|----|----|---|----|----|----|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|
| ord | 0  | 1  | 2 | 3  | 4  | 5  | 6 | 7 | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| pos | 0  | 20 | 1 | 17 | 18 | 11 | 9 | 2 | 19 | 15 | 12 | 10 | 7  | 3  | 16 | 14 | 13 | 8  | 6  | 4  | 5  |
| par | -1 | 0  | 0 | 0  | 0  | 2  | 2 | 2 | 4  | 5  | 5  | 6  | 7  | 7  | 9  | 10 | 10 | 12 | 13 | 13 | 19 |
| hea | 0  | 1  | 0 | 3  | 4  | 5  | 6 | 0 | 4  | 9  | 5  | 6  | 12 | 0  | 9  | 15 | 5  | 12 | 18 | 0  | 0  |

Rastrear: query(2,7)

```
//as consultas nas chains sao sempre de cima para baixo
//note que se o pai de head de v estiver no meio da chain de u --> nao gera problema
//pois o pos de u será menor que v e trocariamos os dois..
//ou seja, sempre teremos pos[u] >= pos[v]
//assim, a consulta é sempre do pos maior para o menor...
T query(int u, int v) { //consulta nos nodos do caminho entre vertices u e v
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) return st.query(pos[v], pos[u]); //estao na mesma chain

    //ATUALIZAR (exemplo: minimo de caminho, maximo de caminho, etc)
    //implementacao atual: soma
    return st.query(pos[head[u]],pos[u]) + query(parent[head[u]], v) ;
}
```



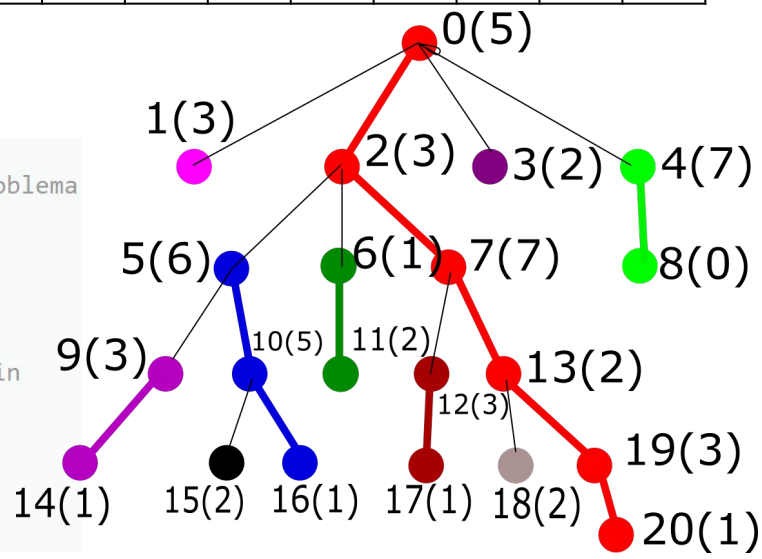
|      |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |    |    |    |
|------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|----|----|----|
| ord  | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| vert | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | 4  | 8  | 1  |

|     |    |    |   |    |    |    |   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----|----|----|---|----|----|----|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|
| ord | 0  | 1  | 2 | 3  | 4  | 5  | 6 | 7 | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| pos | 0  | 20 | 1 | 17 | 18 | 11 | 9 | 2 | 19 | 15 | 12 | 10 | 7  | 3  | 16 | 14 | 13 | 8  | 6  | 4  | 5  |
| par | -1 | 0  | 0 | 0  | 0  | 2  | 2 | 2 | 4  | 5  | 5  | 6  | 7  | 7  | 9  | 10 | 10 | 12 | 13 | 13 | 19 |
| hea | 0  | 1  | 0 | 3  | 4  | 5  | 6 | 0 | 4  | 9  | 5  | 6  | 12 | 0  | 9  | 15 | 5  | 12 | 18 | 0  | 0  |

Rastrear: query(7,10)

```
//as consultas nas chains sao sempre de cima para baixo
//note que se o pai de head de v estiver no meio da chain de u --> nao gera problema
//pois o pos de u será menor que v e trocariamos os dois..
//ou seja, sempre teremos pos[u] >= pos[v]
//assim, a consulta é sempre do pos maior para o menor...
T query(int u, int v) { //consulta nos nodos do caminho entre vertices u e v
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) return st.query(pos[v], pos[u]); //estao na mesma chain

    //ATUALIZAR (exemplo: minimo de caminho, maximo de caminho, etc)
    //implementacao atual: soma
    return st.query(pos[head[u]],pos[u]) + query(parent[head[u]], v) ;
}
```



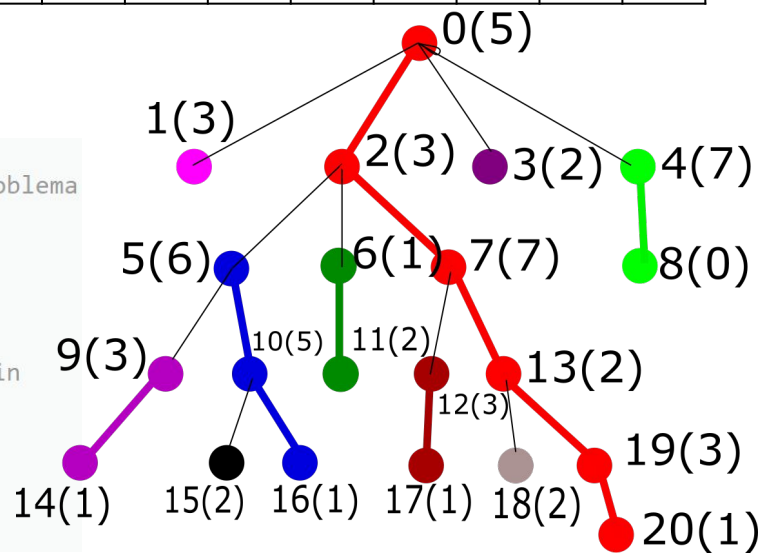
|      |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |    |    |    |
|------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|----|----|----|
| ord  | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| vert | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | 4  | 8  | 1  |

|     |    |    |   |    |    |    |   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----|----|----|---|----|----|----|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|
| ord | 0  | 1  | 2 | 3  | 4  | 5  | 6 | 7 | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| pos | 0  | 20 | 1 | 17 | 18 | 11 | 9 | 2 | 19 | 15 | 12 | 10 | 7  | 3  | 16 | 14 | 13 | 8  | 6  | 4  | 5  |
| par | -1 | 0  | 0 | 0  | 0  | 2  | 2 | 2 | 4  | 5  | 5  | 6  | 7  | 7  | 9  | 10 | 10 | 12 | 13 | 13 | 19 |
| hea | 0  | 1  | 0 | 3  | 4  | 5  | 6 | 0 | 4  | 9  | 5  | 6  | 12 | 0  | 9  | 15 | 5  | 12 | 18 | 0  | 0  |

Rastrear: query(11,16)

```
//as consultas nas chains sao sempre de cima para baixo
//note que se o pai de head de v estiver no meio da chain de u --> nao gera problema
//pois o pos de u será menor que v e trocariamos os dois..
//ou seja, sempre teremos pos[u] >= pos[v]
//assim, a consulta é sempre do pos maior para o menor...
T query(int u, int v) { //consulta nos nodos do caminho entre vertices u e v
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) return st.query(pos[v], pos[u]); //estao na mesma chain

    //ATUALIZAR (exemplo: minimo de caminho, maximo de caminho, etc)
    //implementacao atual: soma
    return st.query(pos[head[u]],pos[u]) + query(parent[head[u]], v) ;
}
```



|      |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |    |    |    |
|------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|----|----|----|
| ord  | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| vert | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | 4  | 8  | 1  |

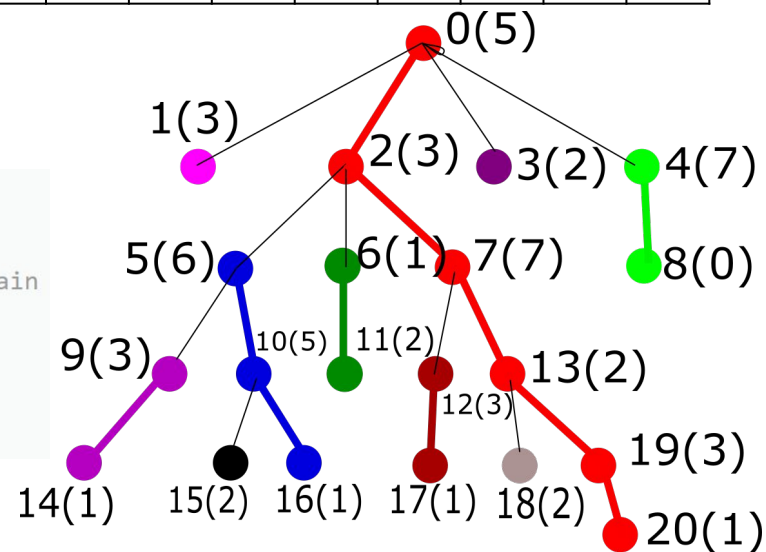
|     |    |    |   |    |    |    |   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----|----|----|---|----|----|----|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|
| ord | 0  | 1  | 2 | 3  | 4  | 5  | 6 | 7 | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| pos | 0  | 20 | 1 | 17 | 18 | 11 | 9 | 2 | 19 | 15 | 12 | 10 | 7  | 3  | 16 | 14 | 13 | 8  | 6  | 4  | 5  |
| par | -1 | 0  | 0 | 0  | 0  | 2  | 2 | 2 | 4  | 5  | 5  | 6  | 7  | 7  | 9  | 10 | 10 | 12 | 13 | 13 | 19 |
| hea | 0  | 1  | 0 | 3  | 4  | 5  | 6 | 0 | 4  | 9  | 5  | 6  | 12 | 0  | 9  | 15 | 5  | 12 | 18 | 0  | 0  |

Update: quase o mesmo código!

No **exemplo** abaixo, soma valor a todos vértices de um caminho

```
//soma valor aos vertices ao longo do caminho..
```

```
void updatePath(int u, int v, T valor) {
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) st.update(pos[v], pos[u], valor); //estao na mesma chain
    else {
        st.update(pos[head[u]], pos[u], valor);
        updatePath(parent[head[u]], v, valor);
    }
}
```

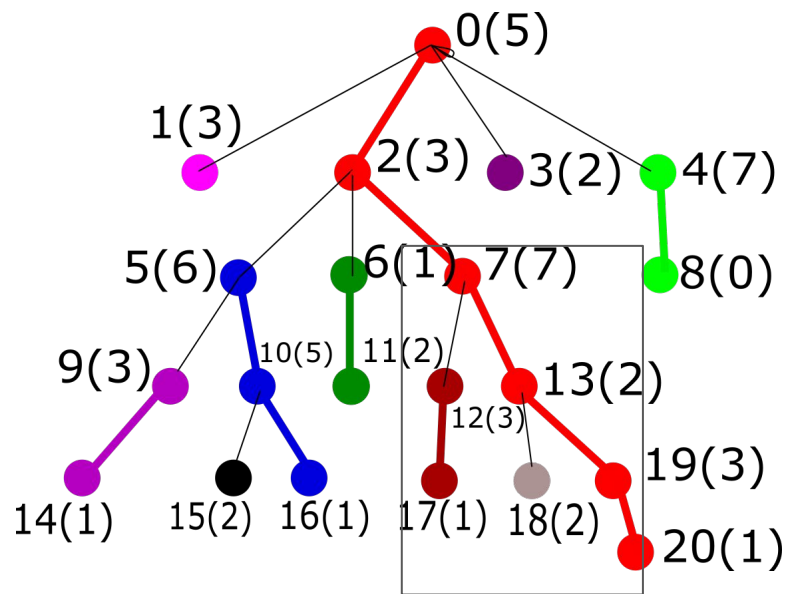


# Bonus

- Fazer operações em subárvores.

|           |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |     |
|-----------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|-----|
| ord       | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | ... |
| vert      | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | ... |
| v (valor) | 5 | 3 | 7 | 2  | 3  | 1  | 2  | 3  | 1  | 1 | 2  | 6  | 5  | 1  | 2  | 3  | 1  | 2  | ... |

- Exemplo: sub-árvore do 7

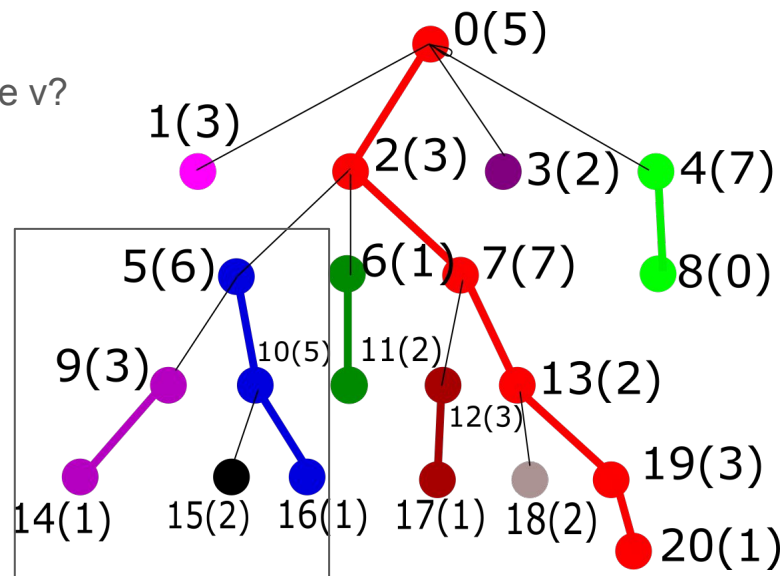


# Bonus

- Fazer operações em subárvores

|           |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |     |
|-----------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|-----|
| ord       | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | ... |
| vert      | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | ... |
| v (valor) | 5 | 3 | 7 | 2  | 3  | 1  | 2  | 3  | 1  | 1 | 2  | 6  | 5  | 1  | 2  | 3  | 1  | 2  | ... |

- Exemplo: sub-árvore do 5
- Em qual intervalo da seg-tree está a sub-árvore do vértice v?

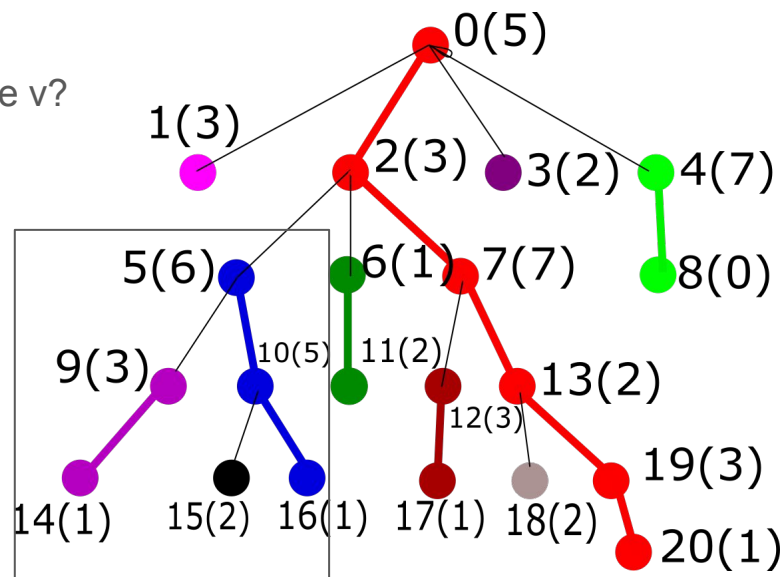


# Bonus

- Fazer operações em subárvores.

|           |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |     |
|-----------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|-----|
| ord       | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | ... |
| vert      | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | ... |
| v (valor) | 5 | 3 | 7 | 2  | 3  | 1  | 2  | 3  | 1  | 1 | 2  | 6  | 5  | 1  | 2  | 3  | 1  | 2  | ... |

- Exemplo: sub-árvore do 5
- Em qual intervalo da seg-tree está a sub-árvore do vértice v?
  - $[\text{pos}[v], \text{pos}[v] + \text{tamArvore}[v] - 1]$



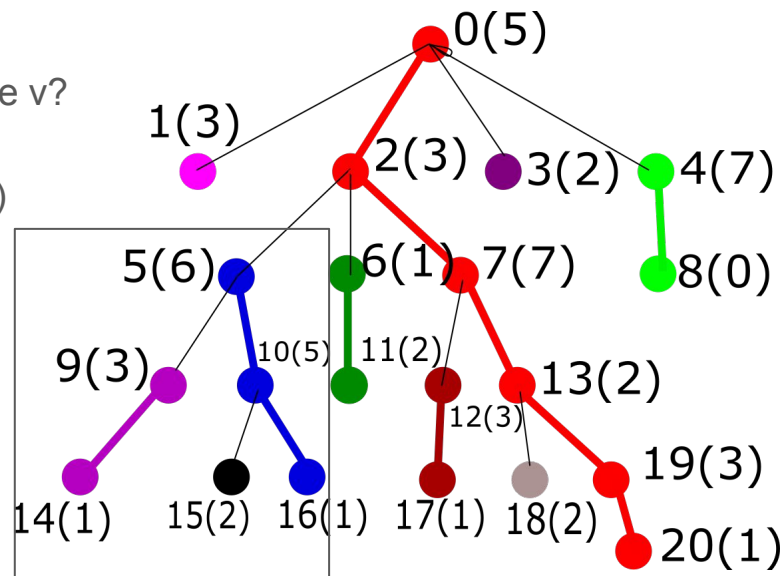
# Bonus

- Fazer operações em subárvores.

|           |   |   |   |    |    |    |    |    |    |   |    |    |    |    |    |    |    |    |     |
|-----------|---|---|---|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|-----|
| ord       | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | ... |
| vert      | 0 | 2 | 7 | 13 | 19 | 20 | 18 | 12 | 17 | 6 | 11 | 5  | 10 | 16 | 15 | 9  | 14 | 3  | ... |
| v (valor) | 5 | 3 | 7 | 2  | 3  | 1  | 2  | 3  | 1  | 1 | 2  | 6  | 5  | 1  | 2  | 3  | 1  | 2  | ... |

- Exemplo: sub-árvore do 5
- Em qual intervalo da seg-tree está a sub-árvore do vértice  $v$ ?
  - $[\text{pos}[v], \text{pos}[v] + \text{tamArvore}[v] - 1]$
  - $\text{tamArvore}$  é o array  $\text{sz}$  (calculado na primeira DFS)

```
//Na dfs, ao processar o nodo u os (sz[u]-1)
//proximos elementos todos de sua subarvore estaraao
//logo apos ele no vetor v de valores da seg tree
T querySubtree(int u) { //consulta na subarvore inteira do nodo u
    return st.query(pos[u], pos[u] + sz[u] - 1);
}
//soma valor a subarvore.
void updateSubtree(int u, T valor) {
    st.update(pos[u], pos[u] + sz[u] - 1, valor);
}
```



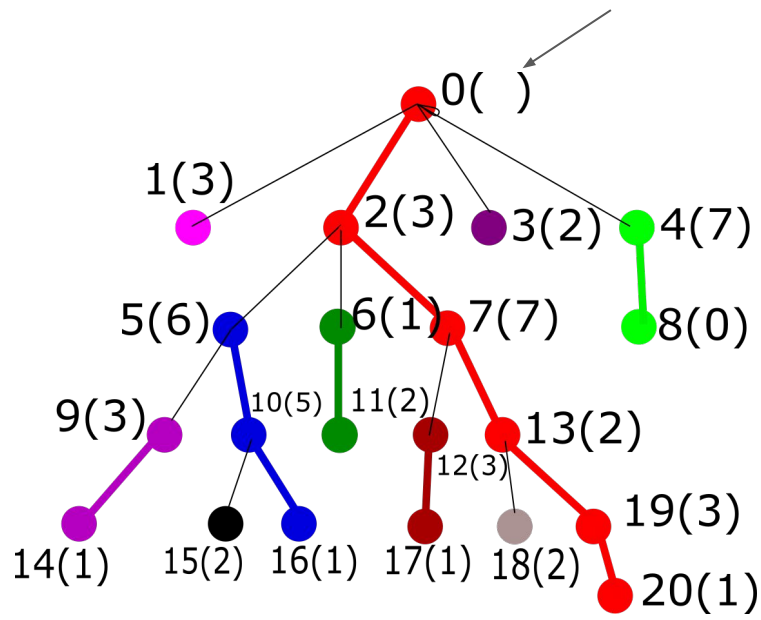
# Bonus

- HLD com valores nas arestas
  - Muda pouca coisa: cada vértice conterá o valor da aresta chegando nele (a partir do pai)
  - Exercício: o código abaixo é para pesos nos vértices. O que muda para pesos nas arestas?

```
//assim, a consulta é sempre do pos maior para o menor...
T query(int u, int v) { //consulta nos nodos do caminho entre vertices u e v
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) return st.query(pos[v], pos[u]); //estao na mesma chain

    //ATUALIZAR (exemplo: minimo de caminho, maximo de caminho, etc)
    //implementacao atual: soma
    return st.query(pos[head[u]], pos[u]) + query(parent[head[u]], v) ;
}

//soma valor aos vertices ao longo do caminho..
void updatePath(int u, int v, T valor) {
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) st.update(pos[v], pos[u], valor); //estao na mesma chain
    else {
        st.update(pos[head[u]], pos[u], valor);
        updatePath(parent[head[u]], v, valor);
    }
}
```



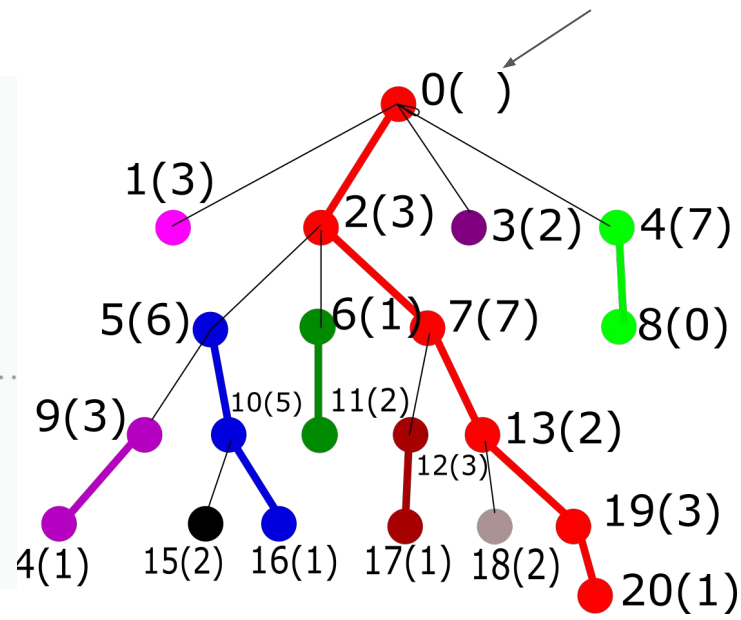
# Bonus

- HLD com valores nas arestas
  - Muda pouca coisa: cada vértice conterá o valor da aresta chegando nele (a partir do pai)
  - A DFS usada na construção é basicamente a mesma (a mudança é copiar os pesos das arestas para os vértices)

```
T query(int u, int v) { //consulta nos nodos do caminho entre vertices u e v
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) return st.query(pos[v]+1, pos[u]); //estao na mesma chain

    //ATUALIZAR (exemplo: minimo de caminho, maximo de caminho, etc)
    //implementacao atual: soma
    return st.query(pos[head[u]],pos[u]) + query(parent[head[u]], v) ;
}

//soma valor ao caminho..
void updatePath(int u, int v, T valor) {
    if(u==v) return; //se o caminho tem apenas 1 vertice --> nao ha aresta para atualizar...
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) st.update(pos[v]+1, pos[u], valor); //estao na mesma chain
    else {
        st.update(pos[head[u]],pos[u], valor);
        updatePath(parent[head[u]], v,valor) ;
    }
}
```



# Complexidade

- $O(n)$  (duas DFS) para construir
- Consulta/update: ?

# Complexidade

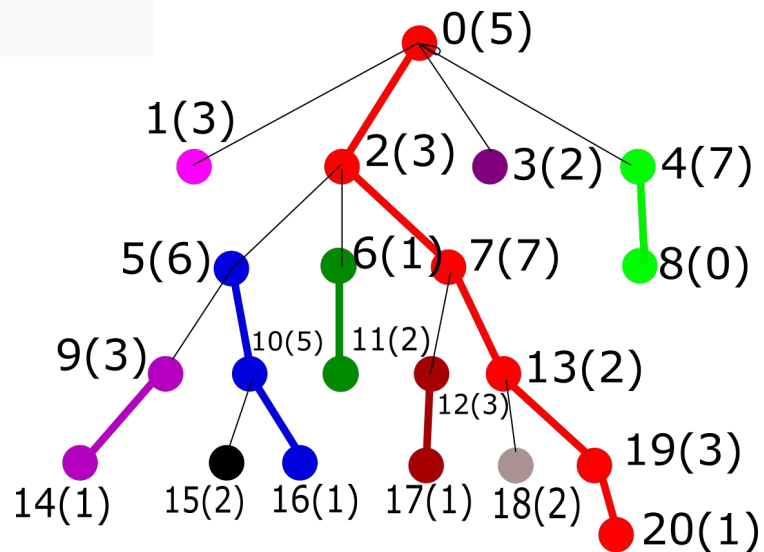
- $O(n)$  (duas DFS) para construir
- Consulta/update:  $O(\log^2 n)$ ,  $O(\log n)$  para o caso de sub-árvores

# Exercícios

- Criar função-membro que calcula o LCA entre dois vértices, adaptando função query

```
T query(int u, int v) { //consulta nos nodos do caminho entre vertices u e v
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) return st.query(pos[v], pos[u]); //estao na mesma chain

    //ATUALIZAR (exemplo: minimo de caminho, maximo de caminho, etc)
    //implementacao atual: soma
    return st.query(pos[head[u]], pos[u]) + query(parent[head[u]], v) ;
}
```



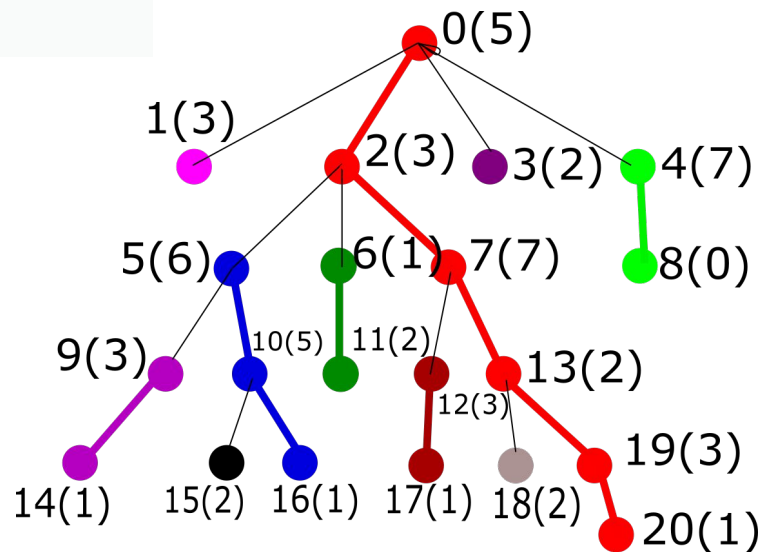
# Exercícios

- Criar função-membro que calcula o LCA entre dois vértices, adaptando função query

```
T query(int u, int v) { //consulta nos nodos do caminho entre vertices u e v
    if(pos[u] < pos[v]) swap(u,v); //posicoes sao de cima para baixo...
    if(head[u]==head[v]) return st.query(pos[v], pos[u]); //estao na mesma chain

    //ATUALIZAR (exemplo: minimo de caminho, maximo de caminho, etc)
    //implementacao atual: soma
    return st.query(pos[head[u]],pos[u]) + query(parent[head[u]], v) ;
}
```

```
int lca(int u, int v) {
    if(pos[u] < pos[v]) swap(u,v);
    //pos[u] >= pos[v]
    if(head[u]==head[v]) //estao na mesma chain...
        return v; //por que é sempre v?
    return lca(parent[head[u]], v);
}
```



# Exercícios

- Teremos exercícios no vjudge.
- Uno (mineira 2024)
  - <https://codeforces.com/group/YgJmumGtHD/contest/528947/problem/L>
  - Talvez passe com HLD?

